

Devoir surveillé terminal

Durée 2h00

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.*

Exercice 1 (Cellula - 10 pts)

Nous souhaitons réaliser une simulation de type jeu de la vie nommée *Cellula*©. Le principe consiste à faire vivre une colonie de cellules dans un environnement donné, que nous modélisons par une grille de taille variable (elle peut contenir des obstacles). Régulièrement, chaque colonie sélectionne une de ses cellules et la fait évoluer en observant son environnement (les 4 cases autour d'elle). Si une de ces cases contient une cellule d'une autre colonie, elle l'absorbe (la case est donc occupée par une nouvelle cellule de la colonie). Sinon, s'il y a de l'espace autour d'elle (une case vide), la cellule se divise sur une des cases choisie aléatoirement.

Chaque colonie de cellules est simulée par une application différente. Elle s'arrête si elle a perdu toutes ses cellules. Il est possible d'en lancer au maximum N . L'application principale, nommée *Pétri*, prend en arguments la valeur de N , la largeur et la hauteur de la grille. Elle crée et initialise les outils IPC nécessaires.

File de messages

Une file de messages IPC est mise en place pour communiquer entre l'application *Pétri* et les colonies de cellules. Lorsqu'une colonie démarre, elle envoie un message à l'application *Pétri* qui lui donne l'autorisation ou non de démarrer. Lorsque l'application *Pétri* s'arrête, elle envoie un message aux colonies pour qu'elles s'arrêtent et attend une réponse de chacune d'elles, avant de s'arrêter à son tour. Enfin, si une colonie s'arrête (via une action de l'utilisateur ou si elle ne possède plus de cellules), elle envoie un message à l'application *Pétri* pour la prévenir.

- 1°) Y'a-t-il des problèmes de synchronisation lors de l'écriture dans une file de messages ?
- 2°) Quelles sont les données échangées dans la file de messages (n'oubliez pas de préciser les types) ?
- 3°) Donnez la structure du message de connexion envoyé par la colonie, puis donnez le code permettant d'envoyer ce message (la clé de la file est fixée par la constante CLE). Déclarez et initialisez chaque variable nécessaire.

Segment de mémoire partagé

Le segment de mémoire partagé permet de partager les données entre l'application *Pétri* et les colonies de cellules. Il contient la grille de taille variable et ses dimensions, et le nombre de cellules de chaque colonie (ce nombre est mis-à-jour par la colonie ou les autres colonies).

- 4°) Proposez une structure permettant de mapper le segment.
- 5°) Écrivez la fonction permettant de mapper le segment (en supposant qu'il existe). Sa signature est la suivante (la clé du segment est fixée dans la constante CLE) :

`segment_t* mapper()`

Sémaphores

6°) Proposez une solution pour gérer les problèmes d'accès concurrents au segment de mémoire partagé, tout en permettant autant que possible une exécution parallèle des différentes applications.

7°) Supposons qu'une colonie désire faire évoluer la cellule située sur la case (x, y) . Expliquez comment réaliser cela tout en gérant les problèmes de synchronisation.

8°) Comment les colonies peuvent-elle détecter leur fin ? À quels moments doivent-elles faire cette vérification ?

Exercice 2 (Cheapster - 10 pts)

Nous souhaitons développer une application de partage de fichiers en mode pair-à-pair nommée *Cheapster*©. Les fichiers sont découpés en blocs de taille fixe (1024o) et partagés entre les clients. Un serveur attend des requêtes UDP de la part de clients. Ceux-ci peuvent envoyer un enregistrement de fichier en envoyant simplement son nom. Le serveur conserve la liste de tous les fichiers et les adresses IP des clients associés. Les clients peuvent également envoyer une demande de fichier. Dans ce cas, le serveur envoie une liste aléatoire d'adresses IP de taille fixe parmi celles associées au fichier. Lorsqu'un client la reçoit, il peut établir une connexion TCP vers les clients pour leur demander la liste des blocs qu'ils possèdent, puis leur demander de télécharger des blocs spécifiques. Les numéros de port sont fixés dans l'application et identiques sur tous les acteurs.

1°) À votre avis, est-ce que le choix d'UDP pour les requêtes entre le serveur et les clients est appropriée ? Comment peut-on gérer la perte de données ?

2°) Illustrez l'architecture de l'application sous la forme d'un schéma (le serveur, 2 clients), en précisant les types de communication et les sockets.

3°) Donnez un diagramme d'échange (un chronogramme) permettant d'illustrer les échanges en partant de la récupération de la liste des clients jusqu'à la récupération d'un bloc du fichier depuis un autre client. Précisez les données échangées (ainsi que leur type).

4°) Expliquez comment un client peut récupérer simultanément 2 blocs d'un même fichier depuis 2 clients différents. Combien de sockets sont nécessaires ?

5°) Comment le serveur peut-il récupérer l'adresse IP des clients lors de l'enregistrement d'un fichier ? Donnez les appels systèmes nécessaires.

Nous supposons que le client exécute la fonction `echange` dans un fils donné. Elle crée la socket qui permet d'attendre la connexion des autres clients. Chaque client qui se connecte est géré dans un fils spécifique (le code correspond à la fonction `fils`).

6°) Donnez le code de la fonction `echange` (le numéro de port TCP est fixé dans la constante `PORT_TCP`). Précisez les paramètres nécessaires pour la fonction `fils`.

7°) Un fichier étant récupéré bloc par bloc, dans un ordre aléatoire, expliquez comment stocker les données (données du fichier et numéros des blocs récupérés) dans un/des fichier(s).

Annexes : quelques en-têtes de fonctions C

```

int          accept(int sockfd, struct sockaddr *adresseClient, socklen_t *taille);
unsigned int  alarm(unsigned int nb_secondes);
int          atexit(void (*procedure)(void));
int          bind(int fd, struct sockaddr *adresse, socklen_t longueur);
int          close(int fd);
int          connect(int sockfd, struct sockaddr *adresseServeur, socklen_t taille);
int          execve(const char *fichier, char *const argv[], char *const envp[]);
void         exit(int statut);
int          fcntl(int fd, int commande, ...);
pid_t        fork();
key_t        ftok(char *chemin, int proj_id);
pid_t        getpid();
pid_t        getppid();
int          getpeername(int sockfd, struct sockaddr *adresse, socklen_t *taille);
int          getsockname(int sockfd, struct sockaddr *adresse, socklen_t *taille);
uint32_t     htonl(uint32_t hote_long);
uint16_t     htons(uint16_t hote_court);
const char * inet_ntop(int famille, const void *source, char *destination, socklen_t taille);
int          inet_pton(int famille, const char *source, void *destination);
int          kill(pid_t pid, int numero_signal);
int          listen(int sockfd, int taille);
off_t        lseek(int fd, off_t offset, int point_depart);
int          lstat(const char *chemin, struct stat *tampon);
void         *memcpy(void *destination, const void *source, size_t taille);
void         *memset(void *tampon, int caractere, size_t nombre_elements);
int          mkfifo(const char *nomFichier, mode_t mode);
int          msgctl(key_t cle, int commande, struct msqid_ds *buf);
int          msgget(key_t cle, int options);
int          msgrcv(int msqid, struct msgbuf *msg, size_t taille, long type, int options);
int          msgsnd(int msqid, struct msgbuf *msg, size_t taille, int options);
uint32_t     ntohl(uint32_t reseau_long);
uint16_t     ntohs(uint16_t reseau_court);
int          open(const char *chemin, int options, mode_t mode);
int          pause(void);
int          pipe(int tube[2]);
ssize_t      recvfrom(int sockfd, void *message, size_t taille, int flags,
                     struct sockaddr *adresse_source, socklen_t *taille_adresse);
ssize_t      read(int fd, void *tampon, size_t taille);
int          semctl(int semid, int semnum, int commande, union senum args);
int          semget(key_t cle, int nbSems, int options);
int          semop(int semid, struct sembuf *sops, unsigned nsops);
ssize_t      sendto(int sockfd, const void *message, size_t taille, int flags,
                     const struct sockaddr *adresse_dest, socklen_t taille_adresse);
void         *shmat(int shmid, const void *shmaddr, int options);
int          shmctl(key_t cle, int commande, struct shmid_ds *buf);
int          shmdt(const void *shmaddr);
int          shmget(key_t cle, int taille, int options);
int          sigaction(int num, const struct sigaction *action, struct sigaction *old_action);
int          sigaddset(sigset_t *ensemble, int numero_signal);
int          sigdelset(sigset_t *ensemble, int numero_signal);
int          sigemptyset(sigset_t *ensemble);
int          sigfillset(sigset_t *ensemble);
int          sigismember(sigset_t *ensemble, int numero_signal);
int          sigpending(sigset_t *ensemble);
int          sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int          sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int          sigsuspend(const sigset_t *ensemble);
int          sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int          sigtimedwait(const sigset_t *set, siginfo_t *info, const struct timespec *timeout);
unsigned int  sleep(unsigned int nombre_secondes);
int          socket(int domaine, int type, int protocole);
int          socketpair(int domaine, int type, int protocol, int sv[2]);
int          unlink(const char *nomFichier);
pid_t        wait(int *statut);
pid_t        waitpid(pid_t pid, int *statut, int options);
ssize_t      write(int fd, const void *tampon, size_t taille);

```

Constantes/macros associées à des paramètres ou des champs de structures

cle (msgget, shmget, semget) : IPC_PRIVATE
commande (fcntl) : F_GETFL, F_SETFL
commande (msgctl, shmctl) : IPC_RMID, IPC_STAT, IPC_SET
commande (semctl) : IPC_RMID, IPC_STAT, IPC_SET, IPC_GETVAL, IPC_GETALL, IPC_SETVAL, IPC_SETALL
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
domaine (socket, socketpair) : AF_LOCAL, AF_UNIX, AF_INET, AF_INET6, AF_PACKET
famille (inet_pton, inet_ntop) : AF_INET, AF_INET6
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
mode (open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (fcntl, open) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
options (msgget, semget, shmget) : IPC_CREAT, IPC_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR...
options (msgrcv) : IPC_NOWAIT, MSG_EXCEPT
options (msgsnd) : IPC_NOWAIT, MSG_NOERROR
options (shmat) : SHM_RDONLY, SHM_RND
options (waitpid) : WNOHANG, WUNTRACED
point_depart (lseek) : SEEK_SET, SEEK_CUR, SEEK_END
protocole (socket, socketpair) : IPPROTO_TCP, IPPROTO_UDP
sa_flags (champ de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
sa_handler (champ de struct sigaction) : SIG_IGN, SIG_DFL
sem_flag (champ de struct sembuf) : IPC_NOWAIT, SEM_UNDO
taille (inet_ntop) : INET_ADDRSTRLEN, INET6_ADDRSTRLEN
type (socket, socketpair) : SOCK_STREAM, SOCK_DGRAM

Macros sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG

Quelques structures C

<pre> struct sigaction { void (*sa_handler) (int); void (*sa_sigaction) (int, sigset_t sa_mask; int sa_flags; }; union semun { int val; struct semid_ds *buf; unsigned short *array; }; struct siginfo_t { int si_signo; int si_code; sigval_t si_value; pid_t si_pid; ... }; struct msqid_ds { struct ipc_perm msg_perm; time_t msg_stime; time_t msg_rtime; time_t msg_ctime; msgnum_t msgnum; msglen_t msg_qbytes; pid_t msg_lspid; pid_t msg_lrpid; }; </pre>	<pre> struct in_addr { uint32_t s_addr; }; struct timespec { long tv_sec; long tv_nsec; }; struct semid_ds { struct ipc_perm sem_perm; time_t sem_otime; time_t sem_ctime; unsigned short sem_nsems; }; struct shmid_ds { struct ipc_perm msg_perm; size_t shm_segsz; time_t shm_atime; time_t shm_dtime; time_t shm_ctime; pid_t shm_cpid; pid_t shm_lpid; shmatt_t shm_nattch; }; struct sockaddr_in { sa_family_t sin_family; in_port_t sin_port; struct in_addr sin_addr; }; </pre>	<pre> struct sembuf { unsigned short sem_num; short sem_op; short sem_flg; }; struct sockaddr_in6 { sa_family_t sin6_family; in_port_t sin6_port; uint32_t sin6_flowinfo; struct in6_addr sin6_addr; uint32_t sin6_scope_id; }; struct sockaddr_un { sa_family_t sun_family; char sun_path[108]; }; struct in6_addr { unsigned char s_addr6[16]; }; union sig_val_t { int sival_int; void *sival_ptr; }; struct timeval { long tv_sec; long tv_usec; }; </pre>
---	--	---